

Models and Database

Replace fixed data with persistent MySQL records.

Learning Goals

- Generate a model and a migration, and adapt them to the existing `students` table.
- Write controller actions that read and save records with the model and validation.
- Check and undo migrations safely on a shared database.

So far, we have focused on the `View` and `Controller` parts of MVC by returning JSON responses.

- In this section, we connect a `Model` to MySQL to store and work with student records.
- We reuse the `studentdb` database from Module 1, so PHP/PDO and Laravel share the same data.

One Database, Two Ways In

	Module 1 (PDO)	This lesson (Laravel)
Database	<code>studentdb</code>	<code>studentdb</code>
Account	<code>ase230 / ase230pass</code>	<code>ase230 / ase230pass</code>
Table	<code>students</code>	<code>students</code>
Columns	<code>id, name, email, age, major, year</code>	<code>id, name, email, age, major, year</code>
Code	<code>\$pdo->prepare("INSERT ...")</code>	<code>Student::create(\$data)</code>

- Only the code that talks to MySQL changes. The data stays

The `make:model Student -m` Command

```
php artisan make:model Student -m
```

Part	Meaning
<code>php</code> <code>artisan</code>	Laravel's command-line tool
<code>make:model</code>	A generator : it writes a starter file for you
<code>Student</code>	The class name. Laravel expects the table <code>students</code> (singular class, plural table)
<code>-m</code>	Also generate a migration

It creates two files and prints something like:

- Student model: app/Models/Student.php
- Database table: database/migrations/...php

```
INFO Model [app/Models/Student.php] created successfully.  
INFO Migration  
[database/migrations/2026_09_29_230058_create_students_table.php]  
created successfully.
```

The command only writes files. It does **not** touch the database.

The Generated Model

app/Models/Student.php :

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Student extends Model
{
    //
}
```

- This is an **ORM** (Object-Relational Mapping): a table maps to a class, and a row maps to an object. Laravel's ORM is called **Eloquent**, so `Student` is an Eloquent model.
- `extends Model` gives you `Student::all()`, `Student::create()`, and `Student::find()` without writing SQL.
- The class body is empty: Laravel guesses the table name `students`, and you add `$timestamps` and `$fillable` yourself (see below).
- An Eloquent model does not automatically validate incoming requests.

The Generated Migration

A migration is a PHP file that describes a change to the database structure, such as creating or modifying a table.

- Laravel applies the change with `up()` and can undo it with `down()`.
- The timestamp in the filename sets the order: Laravel runs older migrations first.

```
database/migrations/2026_09_29_230058_create_students  
_table.php :
```

```

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('students', function (Blueprint $table) {
            $table->id();
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('students');
    }
};

```

- `up()` builds the table (`php artisan migrate`). `down()` undoes it.
- Only `id` and `timestamps` exist. Next, you replace them with the columns Module 1 already uses.

Turn the Scaffold into an Application

Artisan creates starter files, but they are not yet configured for our student database or API.

- We will adapt the migration and model to work with the existing `students` table.
- Then we will implement the controller and routes that expose student records.

- We'll define the columns for the `students` table.
- Then we will build the rest of the flow step by step:
 - i. Match the columns of the `students` table that Module 1 created.
 - ii. Point Laravel's `.env` at the same database (`studentdb`).
 - iii. Run the migration (it skips the table if Module 1 already made it).
 - iv. Add `$timestamps` and `$fillable` to the model.
 - v. Generate the controller and fill in `index`, `show`, and `store`.
 - vi. Add validation before saving a record.
 - vii. Register the routes in `routes/api.php`.

Match the Module 1 Table

Module 1 (`command.txt`) already created this table:

```
CREATE TABLE IF NOT EXISTS students (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100),  
  email VARCHAR(150),  
  age INT,  
  major VARCHAR(100),  
  year INT  
);
```

Our Laravel table must have the same columns. Compare it with the generated migration:

- There is an `age` column.
- There are **no** `created_at` / `updated_at` columns, so the model needs `$timestamps = false`.
- `email` has no UNIQUE index. Laravel's validation rule `unique:students,email` checks duplicates instead.

Define the Table

Inside the generated migration's `up()` method:

```
public function up(): void
{
    if (Schema::hasTable('students')) {
        return; // Module 1 already created it
    }

    Schema::create('students', function (Blueprint $table) {
        $table->integer('id', true); // INT AUTO_INCREMENT PRIMARY KEY
        $table->string('name', 100)->nullable();
        $table->string('email', 150)->nullable();
        $table->integer('age')->nullable();
        $table->string('major', 100)->nullable();
        $table->integer('year')->nullable();
    });
}
```

`Schema::hasTable()` protects the shared table: the migration never overwrites Module 1's data.

Point .env at the Same Database

Laravel reads the connection settings from `.env`. Compare them with `crud_pdo.php` :

<code>crud_pdo.php</code>	Laravel <code>.env</code>
<code>mysql:</code> in the DSN	<code>DB_CONNECTION=mysql</code>
<code>host=localhost</code>	<code>DB_HOST=127.0.0.1</code>
<code>dbname=studentdb</code>	<code>DB_DATABASE=studentdb</code>
<code>"ase230"</code>	<code>DB_USERNAME=ase230</code>
<code>"ase230pass"</code>	<code>DB_PASSWORD=ase230pass</code>

Same database, same account: only the place where you write the settings is different.

Give the Account Migration Rights

Module 1 gave `ase230` basic data privileges (`SELECT` , `INSERT` , `UPDATE` , `DELETE`).

- These allow reading and changing rows, but not creating tables.
- `php artisan migrate` changes the database structure and records completed migrations in Laravel's `migrations` table, so it needs additional privileges.

In the MySQL administrator shell:

```
GRANT ALL PRIVILEGES ON studentdb.* TO 'ase230'@'localhost';
```

- The grant is limited to `studentdb`: the account still cannot touch other databases.

Now run **only** the students migration:

```
php artisan migrate --path=database/migrations/<timestamp>_create_students_table.php
```

- `--path` runs a single file.
- A plain `php artisan migrate` would also add Laravel's default tables (`users` , `cache` , `jobs`) to `studentdb` .

Check a Migration

```
php artisan migrate:status
```

- `migrate:status` shows which migration files have already run.

Undo a Migration

```
php artisan migrate:rollback
```

- `migrate:rollback` runs `down()` for the latest **batch**, possibly several files.
- Our `down()` uses `dropIfExists('students')`: it drops the table **and its data**. Rollback reverses the structure; it does not restore deleted records.
- Do not roll back on the shared `studentdb`. Read `migrate:status` first and only explain the effect.

Allow the Fields You Will Save

We can specify the fields that will be saved in the Model.

```
namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Student extends Model
{
    public $timestamps = false;

    protected $fillable = [
        'name', 'email', 'age', 'major', 'year'
    ];

    protected $casts = [
        'age' => 'integer', 'year' => 'integer',
    ];
}
```

- `$timestamps = false`: the Module 1 table has no `created_at` / `updated_at`.
- `$fillable` controls which fields Laravel assigns; validation checks whether their values are valid.
 - For example, with `$fillable = ['name', 'age']`, `Student::create(['name' => 'Mina', 'age' => 20, 'id' => 99])` assigns `name` and `age`, but not `id`.
 - `$guarded = []` allows every field, so use it carefully.
- `$casts` tells Eloquent to convert an attribute to a chosen type, for example `'age' => 'integer'`.

Create the Controller

```
php artisan make:controller Api/StudentController --api
```

- `Api/StudentController` : creates `app/Http/Controllers/Api/StudentController.php`.
The subfolder `Api/` becomes part of the namespace: `App\Http\Controllers\Api`.
- `--api` : adds the five empty methods an API needs. The form methods (`create` , `edit`) are left out because we return JSON.

```
class StudentController extends Controller
{
    public function index() { }
    public function store(Request $request) { }
    public function show(string $id) { }
    public function update(Request $request, string $id) { }
    public function destroy(string $id) { }
}
```

The generated class is shortened here. Next, you fill in the bodies: `show(string $id)` becomes `show(Student $student)`.

The generated methods are empty, so we need to add the logic for reading records and returning API responses.

- Use the `Student` model to read records from MySQL. For one record, route model binding provides the `$student` instance.
- Return the model or collection as JSON with `response() -> json(...)`.

```
Student::count()  
$student->delete();  
response()->json(...)
```

Read Records APIs

Import the model `App\Models\Student` in your API controller.

```
public function index()  
{  
    return response()->json(Student::all());  
}  
  
public function show(Student $student)  
{  
    return response()->json($student);  
}
```

- Laravel's route model binding uses the `{student}` value in the URL to find a `Student` record and pass it to `$student`.
- If no matching record exists, Laravel returns a 404 response.

```
Route::get('/students/{student}',  
    [StudentController::class, 'show']);  
Route::put('/students/{student}',  
    [StudentController::class, 'update']);
```

The APIs for Save

Import `Illuminate\Http\Request` to process the `request` .

```
public function store(Request $request)
{
    $data = $request->validate([
        'name' => 'required|string|max:100',
        'email' => 'required|email|max:150|unique:students,email',
        'age' => 'required|integer|between:1,120',
        'major' => 'required|string|max:100',
        'year' => 'required|integer|between:1,4',
    ]);
    return response()->json(Student::create($data), 201);
}
```

- We must validate the request.
- The `max` values match the column sizes (`VARCHAR(100)` and `VARCHAR(150)`).
- These example rules must be adapted to your project's data contract.

Update and Delete

The complete lesson code includes these actions:

```
$student->update($data); // Use validated update data  
$student->delete();
```

- Define whether updates require all fields or allow partial changes.
- Unique-email validation should ignore the current student's record.
- Confirm changes with a follow-up GET.

Register the Implemented Actions

In `routes/api.php`, import Route and the API StudentController:

```
Route::apiResource('students', StudentController::class)
    ->only(['index', 'show', 'store']);
```

Request	Method
GET /api/students	index
GET /api/students/{student}	show
POST /api/students	store

Add `update` and `destroy` when you implement them.

New Artisan Command Summary

Create the Laravel project, enable API routing, and generate the API controller:

```
php artisan make:model Student -m  
php artisan migrate
```